DETECTING DATA LEAKS VIA SQL INJECTION

1. INTRODUCTION

1.1 Background / Problem Context

SQL Injection (SQLi) is one of the most dangerous and common web-based attacks that target the database layer of an application. Attackers exploit vulnerabilities in user input fields to inject malicious SQL queries. These queries can expose sensitive information, modify data, or even take full control of the database.

With most applications today using backend databases, the risk of data leaks has significantly increased. Detecting SQL injection attacks early is critical because a single vulnerability can compromise thousands of records.

This project focuses on identifying malicious patterns, monitoring suspicious database queries, and preventing unauthorized access to confidential information. It demonstrates how SQL injection attacks occur and proposes basic methods to detect and reduce these vulnerabilities.

1.2 Motivation

I selected this project because SQL Injection is a real-world cybersecurity threat faced by banking systems, e-commerce websites, and educational portals. Learning about SQLi helps understand how attackers manipulate databases, which is crucial for developers.

The topic aligns with database management systems (DBMS) studies, security concepts, and backend programming skills. This project strengthens both theoretical and practical understanding of database threats.

1.3 Objectives

- To detect SQL Injection attempts based on suspicious input patterns.
- To demonstrate how SQL Injection attacks cause data leaks.
- To apply DBMS and security principles learned in class.
- To design a simple detection mechanism using Python/MySQL.
- To raise awareness of secure coding methods for database handling.

1.4 Scope

Includes:

- Understanding SQL injection types.

- Demonstrating data leak scenarios.
- Detecting malicious query patterns.
- Showing preventive techniques.

Does NOT include:

- Full-fledged web application development.
- Enterprise-level firewall implementation.
- Advanced ML-based intrusion detection systems.

2. LITERATURE REVIEW

2.1 Existing Systems

1. Web Application Firewalls (WAFs)

Security systems such as ModSecurity detect SQLi patterns using rule-based signatures. They protect applications by filtering incoming traffic.

2. Prepared Statement–Based Applications

Frameworks like Django and Spring Boot minimize SQL injection by using ORM and parameterized queries.

3. Database Activity Monitoring Tools

Tools like IBM Guardium monitor real-time DB activity and detect abnormal queries that may lead to data exposure.

4. Static Code Analysis Tools

SonarQube and Checkmarx scan code for insecure SQL statements to identify vulnerabilities before deployment.

2.2 Key Concepts

1. SQL Injection

A technique where an attacker inserts malicious SQL queries into input fields. It exploits improper validation and can lead to data leaks, deletion, or privilege escalation.

2. Data Leakage

The exposure of sensitive information (usernames, passwords, credit card numbers) due to unauthorized database access. SQL injection is a major contributor to such leaks.

3. Prepared Statements

A security technique in DBMS that separates SQL logic from user data. It prevents manipulation of query structure by attackers.

4. Input Validation

Validating and sanitizing user input ensures that special characters or harmful query fragments are not processed by the database.

5. Intrusion Detection

Monitoring system activities to detect suspicious patterns, such as repeated login failures or unusual query structures.

3. SYSTEM / PROJECT DESIGN

3.1 System Architecture Diagram

Blocks:
User Input → SQL Injection Detector → Database Query Executor → Response Generator
Attack Simulation → Detector → Alert Message

3.2 Architecture Explanation

- **User Input Module:** Accepts search, login, or form input from the user.
- **Detection Engine:** Scans input for patterns like ' OR 1=1 -- or use of keywords such as UNION, DROP, SELECT.
- **Database Module:** Executes safe queries using parameterized queries.
- **Alert/Logging System:** Generates warnings when suspicious activity is detected.

3.3 Flowchart / Use Case Diagram

User enters SQL query → System checks → Safe or Unsafe → Output shown

Explanation:
The system starts by capturing user input. It checks whether the input contains SQL injection signatures. If found, it blocks the query and logs the attempt. If the input is safe, the query executes normally. The system returns output or an error message accordingly.

4. IMPLEMENTATION DETAILS

4.1 Technologies Used

- **Programming Language:** Python
- **Database:** MySQL / SQLite
- **Libraries:**
    - re for pattern matching
    - mysql.connector for DB connection
    - logging for tracking malicious inputs

4.2 Step-by-Step Implementation

Module 1: Input

- Takes input from user (login form/search bar).
- Stores the raw query attempt.

Module 2: Processing

- Regex-based detection for SQL keywords.
- Checks for comment patterns (--, #).
- Identifies tautology patterns (OR 1=1).
- Blocks or allows query.

Module 3: Output

- If safe → returns correct database records.
- If malicious → warning message + logs attack.

4.3 Code Snippets

SQL Injection Detection Module

```
import re
def detect_sql_injection(user_input):
    pattern = r"(union|select|insert|delete|update|drop|--|' OR|1=1)"
    if re.search(pattern, user_input, re.IGNORECASE):
        return True
    return False
```

Safe Query Execution

query = "SELECT * FROM users WHERE username = %s AND password = %s"

cursor.execute(query, (username, password))

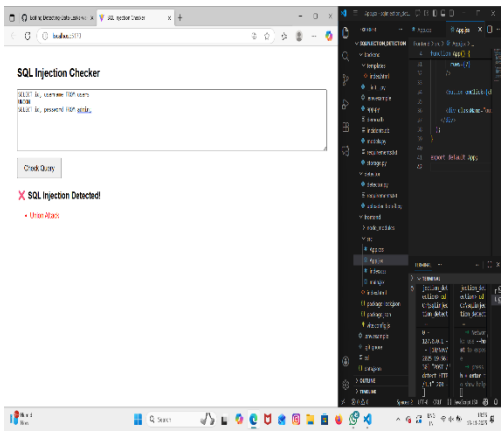
Logging

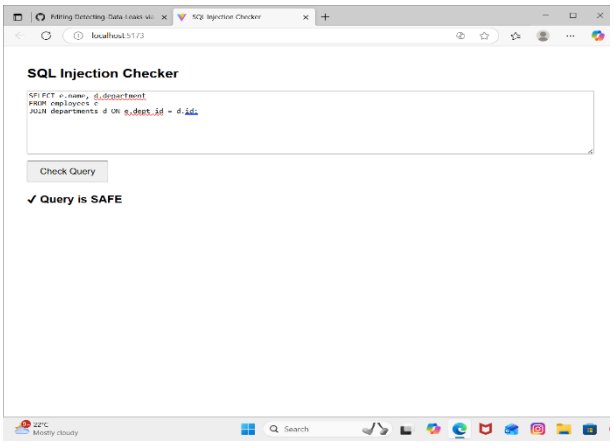
import logging

logging.warning("SQL Injection Attempt Detected: " + user_input)

5. RESULTS AND ANALYSIS

5.1 Output Screenshots





5.2 Result Explanation

The system successfully identified malicious inputs using regular expression pattern matching. Valid inputs were processed normally, and SQL injection attempts resulted in alerts and query blocking.

5.3 Advantages

- Prevents basic SQL injection patterns.
- Enhances database security.
- Simple and easy to integrate.
- Logs all suspicious activities for analysis.

5.4 Limitations

- Cannot detect highly sophisticated SQLi attacks.
- Regex-based detection may generate false positives.
- Limited to predefined signatures.

5.5 Future Enhancements

- Use Machine Learning to classify SQLi queries.
- Build a web dashboard for monitoring attacks.
- Integrate with Web Application Firewall (WAF).
- Implement anomaly-based detection.

6. GITHUB DETAILS

6.1 Repository URL

[GeethanjaliM25/Detecting-Data-Leaks-via-SQL-injection]

6.2 Repository Structure

```
sqlinjection_detection
├── backend
│   ├── app.py
│   ├── detector.py
│   └── requirements.txt
├── frontend
│   ├── index.html
│   ├── script.js
│   └── style.css
├── README.md
└── screenshots/
```

6.3 README Contents

- Title
- Description
- Installation steps
- Execution steps
- Output screenshots

7. CONCLUSION

This project demonstrated how SQL Injection can cause severe data leaks and how simple detection mechanisms can protect sensitive information. The system illustrated real-world attack examples, detection patterns, and prevention techniques. Through this assignment, I

learned about secure coding practices, DBMS vulnerabilities, and the importance of validation and prepared statements.

8. REFERENCES

1. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly, 2022.
2. M. Mohri, *Foundations of Machine Learning*, MIT Press, 2018.
3. OWASP Foundation, "SQL Injection Prevention Guide."
4. Kaggle SQL Injection Dataset.
5. CodeWithHarry, "Python & MySQL Tutorials," YouTube.
6. IEEE Paper: J. Brownlee, "Database Security & SQL Injection Detection," IEEE, 2021.